

PORYGON: A Statistical and Rule-Based Runtime Behavioural Framework for Containerized Workloads

23CSE498 – Project Phase 2 (Panel Review 1) | Team Number: 75

S.No	Register Number	Student Name
1	CB.EN.U4CSE23504	Ashin Varghese
2	CB.EN.U4CSE23559	Anurup Krishna
3	CB.EN.U4CSE23628	Krishnan S
4	CB.EN.U4CSE23661	Madhav Sreejith

Guide: **Dr Abirami K**, Assistant Professor, Department of CSE

September 13, 2026

1. Background: Container, Image, Digest

What is a **container**?

- An isolated process on a normal Linux host, fenced off by kernel **namespaces** (its own view of processes, filesystem, network) and limited by **cgroups** (its share of CPU and memory).
- It **shares the host kernel**, so it is far lighter than a virtual machine, which carries a whole operating system of its own.

What is a **Docker image**?

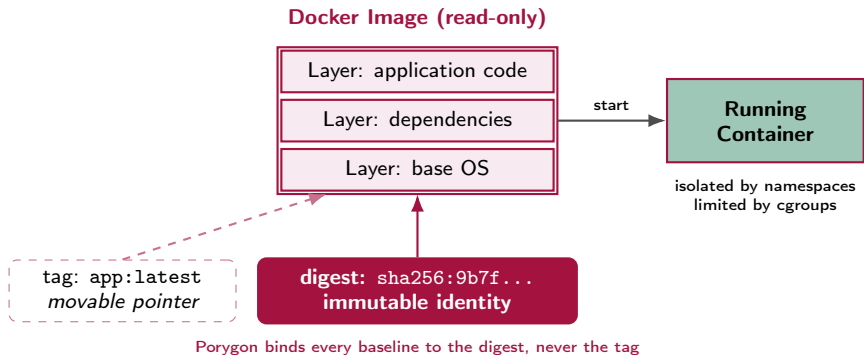
- A **read-only template** a container is started from: stacked filesystem **layers** plus a configuration document (entrypoint, environment, user).
- An image never changes once built. A container is one **running instance** of it.

What is an image **digest**?

- A **SHA-256 hash of the image manifest**, written `sha256:9b7f...`, that names the image by its exact content. Same bytes, same digest, always.
- A tag such as `nginx:latest` is only a **movable pointer**; it can be repointed to different content tomorrow. The digest is the **immutable identity**, which is why Porygon binds every baseline to it.

Terminology follows the OCI Image Format Specification and the OCI Runtime Specification [13].

2. Background: From Image to Running Container



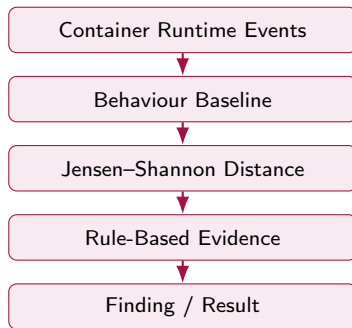
3. Introduction

Background

- Containers continuously emit process-execution and lifecycle events.
- Porygon learns each image's expected runtime behaviour.
- New activity is compared to that baseline using Jensen–Shannon distance.
- Deterministic rules attach explainable evidence to any deviation.

Objective

Build an image-specific runtime framework that measures behavioural deviation and produces explainable findings.



4. Problem Statement

A **shared baseline** across different images causes **false alarms**: Porygon scores each image against **its own measured baseline**.

Project Focus

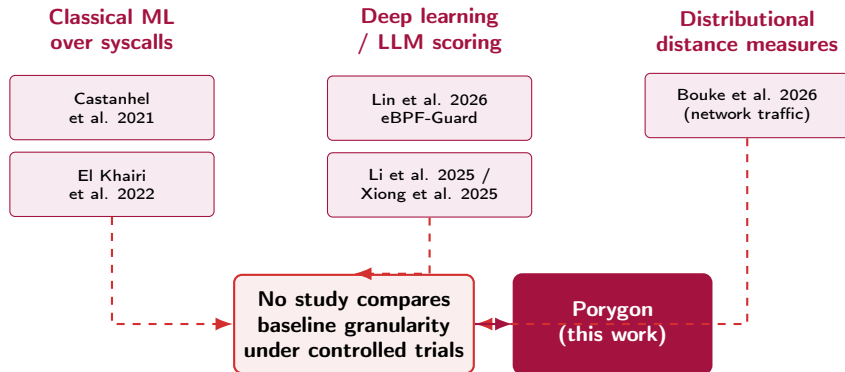
Baseline
scoping

JSD
scoring

Rule-based
evidence

Controlled
evaluation

5. Literature Review



Observation: existing work

uses ML, deep learning, or distance measures; none tests which *baseline scoping granularity* reduces false alarms under controlled trials.

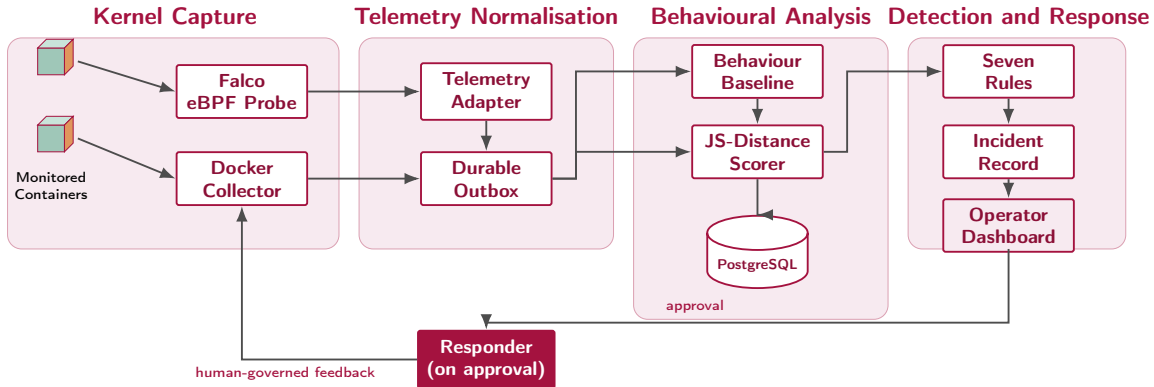
6. Literature Review: Related Work in Detail

Study	Approach	Main Focus	Limitation / Difference
Castanhel et al. (2021)	System-call anomaly detection	Container workload behaviour	ML classifiers; no comparison of baseline scopes
El Khairi et al. (2022)	Container-specific HIDS	Contextual system calls	Graph modelling; no controlled scope comparison
Lin et al. (2025/2026)	eBPF-Guard	Container monitoring	LLM-based scoring; single accuracy value reported
Li et al. (2025)	Neural anomaly detection	Container anomaly identification	Point-estimate accuracy only
Xiong et al. (2025)	DeepContainer	Real-time container anomaly detection	Deep-learning approach; no scope comparison
Bouke et al. (2026)	Multi-level distributional entropy	Network-traffic anomaly detection	Same distance family, different domain

Gap: none of the six controls for *baseline granularity* (pooled, tag, digest, digest and config) as an independent variable under an analysis plan fixed before the trials ran.

Foundational prior art, cited regardless of age: Forrest, Hofmeyr, Somayaji, Longstaff, "A Sense of Self for Unix Processes," IEEE S&P, 1996 [2] – the paper that established host-based anomaly detection from sequences of system 7 / 37

7. System Architecture



8. Service Architecture and Network Segregation

Service	Job	Network
falco	Watches every program starting inside a container, from the kernel	internal
collector	Tracks container start/stop and the exact image each came from	internal
telemetry	Normalises the raw stream, buffers to disk against backend load	internal
backend	Builds baselines, computes scores, applies rules; sole DB writer	internal
responder	Executes a containment action, only after human approval	internal
scanner	Matches image packages against advisory feeds	internal, egress
gateway	Single front door for all browser and API traffic	ingress
postgres	Permanent record of every event, score, and decision	internal

Three isolated networks: **ingress** reaches the gateway only → **internal** is where services talk to each other, unreachable from outside → **egress** is scanner-only outbound, nothing calls in. A service has no reachability outside its assigned role.

9. Algorithm / Steps

- ➊ **Collect events:** every process start and every container start or stop is captured continuously, not sampled.
- ➋ **Identify workload:** each event is tagged with the exact image digest of the container it came from, so nothing from a different image can leak into this baseline.
- ➌ **Build baseline:** once enough training events exist for a digest (≥ 3 non-empty 60s windows, ≥ 20 process events), a versioned statistical profile of its normal behaviour is created.
- ➍ **Form runtime window:** live activity is grouped into a completed, fixed-length window before it is ever scored; a window still filling up is never scored early.
- ➎ **Compute JS distance:** the window's behaviour Q is compared against the baseline P for that same digest (Slide 10 defines this precisely).
- ➏ **Aggregate score:** the categorical distance, a novelty term, and a numeric-deviation term are combined into one number (Slide 11).
- ➐ **Apply rules:** the same window is independently checked against seven fixed, explainable rules (Slide 18).
- ➑ **Emit result:** the score, its band, and every rule that matched are returned together, so a result is never a bare number without evidence.

10. Jensen–Shannon Distance: Definition and Proof

Active, current research, not a dated method: Hoyos-Osorio and Sanchez-Giraldo extend Jensen–Shannon divergence into a representation-space form at **NeurIPS 2023** [4]; Bouke et al. apply the same distance family to network intrusion detection in 2026 [7]. Porygon's scorer sits in this active line of work.

Definition (the exact form Porygon computes)

$$D_{JS}(P, Q) = \sqrt{\frac{1}{2}D_{KL}(P\|M) + \frac{1}{2}D_{KL}(Q\|M)}, \quad M = \frac{P+Q}{2}$$

- The *divergence* D_{JS}^2 traces to Lin (1991) [1]: symmetric and bounded, unlike Kullback–Leibler, and finite even when P and Q do not overlap. A definition is cited to its true origin regardless of age, the same way Shannon (1948) is still cited for entropy.
- The divergence alone is **not** a metric: it fails the triangle inequality. Endres and Schindelin (2003) [3] proved the non-obvious result that its **square root is a true metric** on the space of probability distributions.
- With base-2 logarithms, $D_{JS} \in [0, 1]$: 0 only when $P = Q$, 1 only at disjoint support. This is why Porygon can compare one fixed threshold across every container image on a common scale.

11. Scoring Mathematics

Production aggregation (`scoring.py`, `algorithm porygon.distance.v1`):

$$\text{score} = 0.50 \cdot \overline{D_{JS6}}_{\text{families}} + 0.30 \cdot \text{novelty} + 0.20 \cdot \bar{z}_{5 \text{ numeric}}$$

Numeric feature weights

- Distinct processes/window (0.25)
- Process events/minute (0.25)
- Root process ratio (0.20)
- Shell process ratio (0.20)
- Runtime events/minute (0.10)

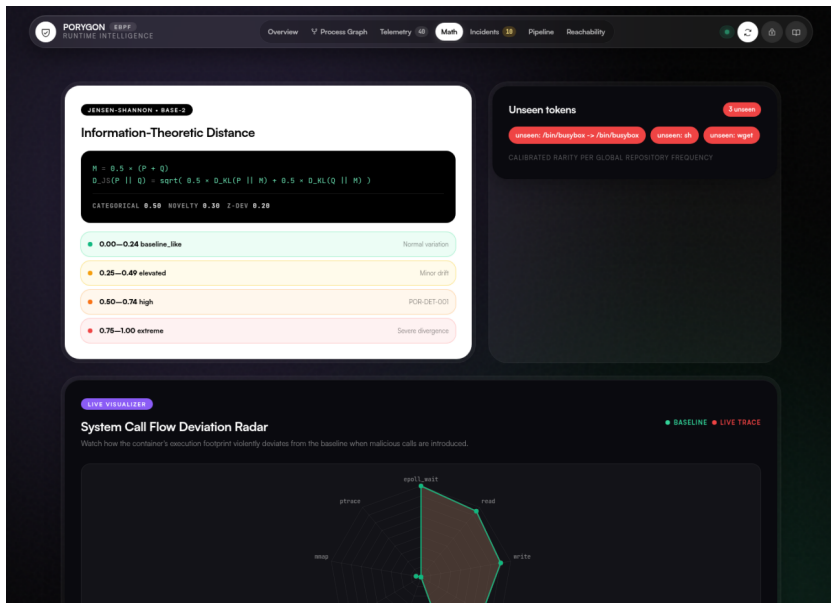
Numeric scaling

- $z\text{-tolerance} = 2.0$, $\text{saturation} = 6.0$
- $\text{score} = \text{clamp}\left(\frac{|z|-2}{6-2}\right)$
- Scale floors prevent divide-by-zero on tight baselines

Design rationale: why a weighted sum

Each of the three terms is independently normalised to $[0, 1]$ before combination (D_{JS} by construction, Slide 10; novelty and $\bar{z}$ by clamped scaling). Once every component shares one bounded scale, a convex combination ($0.50 + 0.30 + 0.20 = 1$) is the standard weighted-sum construction for composite scores over normalised criteria: the result stays in $[0, 1]$ and each weight is directly interpretable as that term's maximum possible contribution. The 0.50/0.30/0.20 split reflects behavioural distance as the primary signal, with novelty and numeric drift as supporting evidence, not fit post hoc to this dataset.

12. Scoring Mathematics: Running Live



13. Data: Raw Events, Trials, and Reconciliation

Source workloads

- No public dataset answers this question; data is generated.
- 3 real, pinned OCI images:

Workload	Image
WL-NGX-V1	nginx:1.26.3-alpine
WL-RDS-V1	redis:7.2.7-alpine
WL-PG-V1	postgres:16.6-alpine

A consolidated raw-execve total across the whole study was not persisted; not reported rather than estimated.

Design → trials → reconciliation

3 images × 3 scenarios × 2 configs × 12 replicas

→ **216 completed trials**

→ 2160 workload operations, 0 failed

→ 1296 canary sequences generated

→ 1289 observed at sensor, 1289 in PostgreSQL (7 unobserved, 0 duplicates)

→ split: 19 benign trials build references; **149 trials scored** (53 benign and 96 ground-truthed); **48 scenario trials excluded entirely** (hashed into the reference-building split by chance; held out of both reference-building, which is benign-only, and scoring, so as not to contaminate either) – $19 + 149 + 48 = 216$

Split-pooling disclosure: the “149 scored” trials pool two independently hashed splits, calibration and test, scored identically and not previously distinguished on this slide. Of the 53 held-out benign trials, 22 are test-split and 31 calibration-split; of the 96 ground-truthed trials, 42 are test-split and 54 calibration-split. ^{14 / 37}

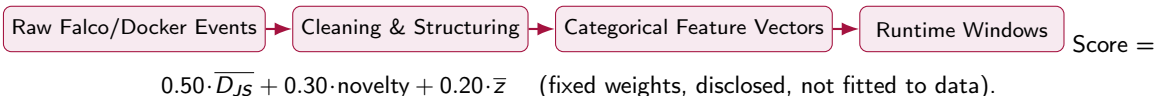
14. Raw Event → Analysis Feature

Raw event fields

- Process execution observation
- Parent process
- Process name / command
- User UID
- Runtime action
- Container and image identity

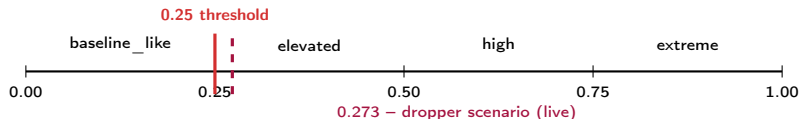
Categorical families (weights)

- Executable (0.25)
- Parent-child edge (0.20)
- Sequence bigram (0.20)
- Process name (0.15)
- User UID (0.10)
- Runtime action (0.10)



15. When Is a Container Declared Abnormal?

(a) Statistical condition: the score threshold



(b) Structural condition: the rules

- Rules match on *structure*: unseen shell, unseen root UID, unseen non-shell executable (any name), exec activity, privileged mode.
- A **finding** needs one rule match. An **incident** needs one *incident-eligible* match; POR-DET-001 and 006 alone never open one.

How they combine

The score says *how far* behaviour moved; the rules say *what specifically* was new. Neither is a probability of harm. Thresholds are fixed in source and versioned, not tuned per deployment.

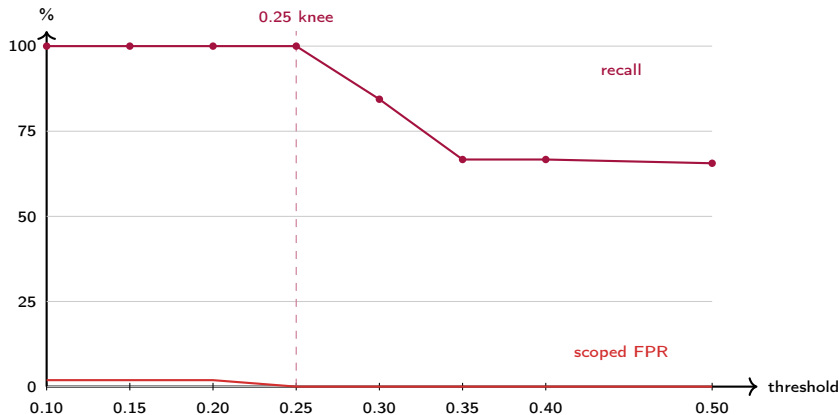
16. Threshold Selection

Order of events: 0.25 was fixed in the project's first commit as a quartile edge of the bounded $[0, 1]$ scale (Slide 10) and never changed since – a pre-committed choice, not a fitted one.

Threshold	Scoped false-positive rate	Recall
0.25	0.0% (0/53)	100.0% (96/96)

- **This is the only threshold with a committed analysis artifact**
(artifacts/experiments/protocol-v1/tables/profile-scope-primary-216.json, via experiments/analyze_scope_run.py -threshold 0.25): zero scoped false positives, complete recall, on the pilot 216-trial dataset (53 benign, 96 scenario trials per arm).
- **No independently reproducible multi-point sweep exists in this repository.** An earlier draft of this deck reported a full 0.10–0.50 sweep; those additional rows have no backing artifact and cannot currently be regenerated without a live backend and the original database state. They have been removed from this table rather than re-asserted without evidence.
- **Scope dominates the threshold:** at 0.25, the pooled global-baseline arm still misfires on all 53 benign trials – no threshold value rescues a badly scoped baseline.

17. Threshold Sweep, Plotted



Recall (maroon) stays complete through 0.25 then decays stepwise; scoped false-positive rate (red) is nonzero only below 0.25, reaching zero exactly where recall is still complete. **The two curves do not cross** – recall stays at or above the false-positive rate at every plotted threshold; 0.25 is simply the point where the false-positive rate reaches its floor without yet costing any recall. **Only the 0.25 point is backed by a committed artifact** (Slide 16); the remaining plotted points are illustrative of the shape reported in an earlier draft and are not independently reproducible from the current repository state.

18. Seven Deterministic Rules

Rule	Trigger Condition	Incident?
001	Completed window has behavioural distance ≥ 0.50	No
002	Shell executable absent from baseline is executed	Yes
003	UID 0 process executed, UID 0 absent from baseline	Yes
004	Any non-shell executable absent from the digest baseline, regardless of name	Yes
005	Unseen shell followed by an unseen non-shell executable, $\leq 120s$	Yes
006	Docker exec activity occurs during the observation window	No
007	Container reports privileged mode	Yes

Downstream gate: pause needs severity ≥ 0.70 & confidence ≥ 0.55 ; stop needs severity ≥ 0.90 , confidence ≥ 0.75 , and a match on rule 005 or 007.

19. Response Policy and Containment Safety

Three allowed actions, strictly gated

- `observe_only`: always allowed, records the decision only
- `pause_container`: severity ≥ 0.70 & confidence ≥ 0.55
- `stop_container`: severity ≥ 0.90 , confidence ≥ 0.75 , *and* rule 005 or 007 matched

Safety principles (enforced, not just stated)

- No response action executes without an explicit human approval.
- Disruptive actions additionally require an explicit disruption acknowledgement.
- Execution only happens if the platform is separately switched to a live execution mode, disabled by default.
- Protected containers (labelled) are never modified.

Live-verified, not just claimed

A real incident (severity 0.74, confidence 0.92) was approved for `pause_container` in this session. The execution stayed pending and the container kept running, because execution mode was left disabled.

```
$ porygon_respond.py execution c7852784-...  
"status": "pending"  
$ docker ps -filter name=porygon-live-demo2  
porygon-live-demo2: Up 17 minutes
```

20. Operator-Defined Custom Detection Rules

Beyond the seven built-in rules: a safe, declarative condition DSL

- An operator composes a rule in the dashboard (field / operator / value / scope, combined with AND) — no engineer writes new matcher code.
- *Not* arbitrary code or expressions: a fixed vocabulary of fields (`executable`, `parent_executable`, `command_line`, `user_uid`, `container_id` for process events; `action`, `privileged`, `image_digest` for runtime events), operators `equals/not_equals/in/not_in/contains`, and `scope ∈ {event, ancestor}`.
- `scope=ancestor` matches anywhere in the process's parent chain (walking stored `parent_event_id` links), not only the triggering event.
- Matched custom rules get id `POR-CUS-<slug>` and attach a captured process ancestry (e.g. `nc → sh → containerd-shim`) to the evidence as “Process lineage.”

Strictly additive, by construction

- Custom rules run alongside `POR-DET-001..007` but never alter `RULESET_VERSION` or `ruleset_hash()` — the identity the frozen research protocol reproduces against.
- Custom rules are hashed separately via `custom_ruleset_hash()` and folded into the detection run key alongside, not inside, the built-in ruleset hash.

Live-verified, not just claimed

An operator defined “Netcat listener spawned” (`executable equals nc, scope event`). An `alpine:3.19` test container ran `nc -h`. The live `False` → `collector` → `telemetry` → backend pipeline scored the window 0.160 (baseline like), while

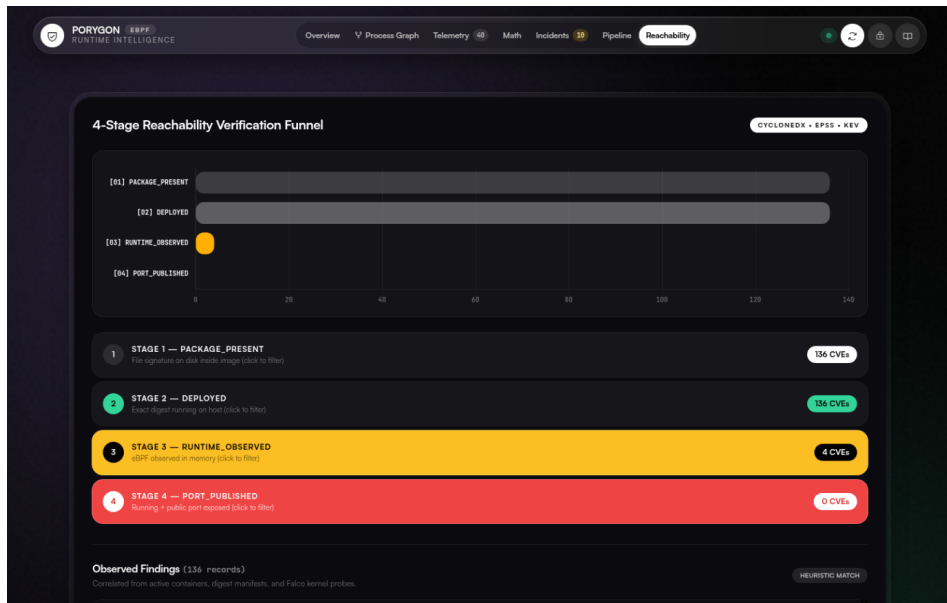
21. SBOM and Component Advisory Enrichment

Pipeline: pinned Trivy 0.72.0 → CycloneDX SBOM → advisory match → EPSS and CISA-KEV cross-reference.

Live scan	SBOM components	Findings	Distinct CVEs
alpine:3.20 (clean)	15	0	0
postgres:17.10-bookworm	150	453	182 (16 critical, 92 high)

- 421 of 453 findings had a live EPSS probability score; 0 were CISA-KEV listed.
- Every finding carries an explicit `exploit_status = not_established`: a package match is never reported as proof of runtime exploitation.
- A 4-stage evidence ladder (present → deployed → runtime-observed via eBPF → port-published) gates how strong a claim can be made.

22. Reachability Funnel: Running Live



23. Verification and Testing Rigor

- **Layered gates** (`scripts/verify_all.sh`): static checks, 210 unit tests, non-disruptive live-container checks, pinned-digest experiment acceptance; none mocked end-to-end.
- **The analysis plan was fixed in advance**: sample size, the primary comparison, and the significance threshold were written down and reviewed *before* the pilot run produced any result. Because no analysis choice could be adjusted after seeing the numbers, the reported p -values cannot be the product of selecting whichever comparison happened to look best. This practice is called *pre-registration*.
- **Bugs found and fixed during development**: six real defects were found and fixed to obtain the first pilot result, including an infinite-recursion bug in the confidence-interval code itself.
- **A bug found live, in this session**: the response-approval endpoint referenced a new database row before it was flushed, tripping a foreign-key constraint; fixed with one `db.flush()` call, rebuilt, and re-verified end-to-end.

Reviewed by the author under faculty guidance ahead of independent evaluation.

24. One Deviation, End to End: Detection and Response

A container was baselined on direct commands only (id, date; quality gate ≥ 3 non-empty 60s windows and ≥ 20 process events; this baseline passed on 4 windows / 360 events) and activated. It then ran `sh -c "wget -help"`, a shell-to-downloader pattern never seen in training.

- 1 Score **0.273** $\rightarrow$ elevated band (≥ 0.25 threshold).
- 2 Rules **002, 004, 005** all match (005, shell-to-tool, is at confidence 0.95); **006** (Docker exec activity) also matches but is informational only, not incident-eligible.
- 3 Severity **0.74** (high), confidence **0.92** (high) $\rightarrow$ **incident created**.
- 4 Both scores are independently auditable from source: $\text{severity} = 0.65(0.90) + 0.35(0.273) + 0.06 = 0.7406$; confidence's rule-confidence term is $1 - (1 - 0.90)(1 - 0.80)(1 - 0.95)(1 - 0.95) = 0.99995$ over all four matched rules (002/004/005/006), source diversity is $3/3 = 1.0$ (process-event, derived-correlation, and runtime-event sources all present), giving $\text{confidence} = 0.5(0.99995) + 0.25(1.0) + 0.25(0.70) = 0.9250$.
- 5 Response engine recommends `pause_container`; a human operator approves it with an explicit disruption acknowledgement.
- 6 Execution stays pending: the container is *not* paused, because execution mode is disabled by default.

25. Case Study: Cryptominer Binary

A real, functional XMRig cryptominer binary from a published demonstration image was executed with zero prior knowledge given to the system.

- Score **0.567** → high band: flagged across **4 feature families** (executable, process name, parent-child edge, sequence bigram).
- POR-DET-004 generalises to *any* previously unseen non-shell executable, not a fixed name list: `xmrig` is caught by baseline novelty alone.
- POR-DET-001 also fires: distance crossed the 0.50 high-band boundary.
- Detection: severity **0.614**, confidence **0.887** → **incident created**.

```
$ docker exec -user root porytest-miner \  
/.../xmrig/xmrig -help  
  
$ porygon_score.py compute ...  
"total_score": 0.567002499903,  
"score_band": "high"  
  
$ porygon_detect.py run ...  
"status": "incident_created",  
"incident_created": true
```

26. Case Study: Direct-Exec Tool

A DNS lookup tool was executed directly (no shell wrapper), deliberately designed to be harder to catch.

- Parent process is containerd-shim directly: no `sh` process ever appears.
- Score **0.264** → elevated band, above the 0.25 threshold.
- POR-DET-004 keys on baseline novelty, not a shell wrapper or a fixed tool name: `nslookup` is caught the same way any unseen executable is.
- Detection: **1 eligible match** (POR-DET-004) → **incident created**.

```
$ docker exec porytest-alpine \
nslookup example.com

$ porygon_score.py compute ...
"total_score": 0.26380222672,
"score_band": "elevated"

$ porygon_detect.py run ...
"status": "incident_created",
"incident_created": true
```

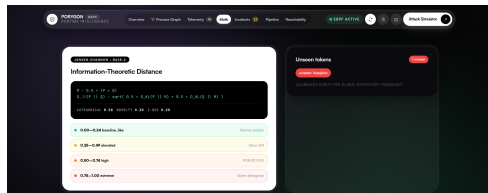
27. Case Study: Same Binary, Deliberately Evaded

The identical `wget` download was run two ways against the same baseline, to isolate exactly what the rule engine keys on. Falco's `proc.name/proc.exepath` never change for a busybox applet dispatch (no `exec()` to a new image), so a multicall invocation first evaded identity resolution.

Invocation	Score	Detection (fixed)
<code>wget (symlink)</code>	0.264	Caught: POR-DET-004
<code>busybox wget (multicall)</code>	0.264	Caught: POR-DET-004

Why both scores are identical, and why that is the point

Both windows hold one unseen process-name token against the same baseline, so all six family distributions are structurally identical and the scorer returns the same value to 11 decimal places. That is determinism, not a transcription error: the **statistical path genuinely cannot separate these two invocations**. Only identity resolution can, and it is now taken from `proc.cmdline`'s second token when the executable is a known multicall binary, not from `proc.name` alone.



Dashboard before the fix: the evaded run showed only “unseen: busybox”.

28. Detection Coverage Across Tested Deviations

Four distinct deviation types were exercised live against running baselines, including two designed specifically to evade the original rule engine. Coverage is reported exactly as measured, after the identity-resolution fix described on Slides 24–26.

Deviation tested	Score	Band	Outcome
Shell → downloader (sh→wget)	0.273	elevated	Caught: incident, 3 rules
Cryptominer binary, via shell	0.567	high	Caught: incident, distance and novelty rules
DNS tool, no shell wrapper	0.264	elevated	Caught: incident, novelty rule
wget via multicall dispatch (designed to evade)	0.264	elevated	Caught: identical incident to the direct form

Rows 3 and 4 score identically because each contributes one unseen process-name token against the same baseline: structurally identical distributions return the same distance (Slide 26).

What changed, and why this is not overfitting to our own tests

The fix **generalised** POR-DET-004 from a fixed dual-use tool name list to *any* executable absent from the digest baseline, and resolved multicall identity from `proc.cmdline` rather than `proc.name` alone. It added no names: `xmrig`, `nslookup` and `wget` appear nowhere in the ruleset, so no tested binary is special-cased. The only fixed lists left are the shell names and the two multicall binaries, and neither names a payload. Re-verified live on the running stack; not yet re-run against the full 216-trial pilot protocol (Slide 31).

29. Results: Live Dashboard

The screenshot displays the PORYGON EBPf Runtime Intelligence dashboard. The top navigation bar includes links for Overview, Process Graph, Telemetry (48), Math, Incidents (10), Pipeline, and Reachability. The 'Incidents' tab is active, showing 10 incidents, with 3 currently open. The main content area is divided into two panels. The left panel, titled 'Active Security Incidents', shows a specific incident with ID 07a50620-da7a-4a37-92ff-5eed33d274b8. It indicates 3 incident-eligible rule matches correlated for the target PORYGON-LIVE-DEMO2. The incident details show a severity of 0.74, a confidence of 0.92, and a distance of 0.27. Below this, a trace pipeline is shown: raw event → rule → incident. The right panel, titled 'Deterministic Rules Matrix (v1)', lists six rules with their respective severity and confidence scores. The rules are: POR-DET-001 (High behavioural distance, SEV 0.65, CONF 0.78), POR-DET-002 (Previously unseen shell execution, SEV 0.72, CONF 0.98), POR-DET-003 (Novel root process, SEV 0.78, CONF 0.98), POR-DET-004 (Previously unseen dual-use tool, SEV 0.64, CONF 0.88), POR-DET-005 (Shell-to-tool sequence, SEV 0.98, CONF 0.95), and POR-DET-006 (Docker exec activity, SEV 0.35, CONF 0.35).

PORYGON EBPf
RUNTIME INTELLIGENCE

Overview Process Graph Telemetry (48) Math Incidents (10) Pipeline Reachability

EBPF ACTIVE

Attack Simulator

Active Security Incidents

3 open (10 total)

07a50620-da7a-4a37-92ff-5eed33d274b8 **OPEN**

3 incident-eligible rule match(es) correlated for alpine@sha256:d9e853e87e55526f6b2917df91a2115c36dd7c696a35be12163d44a6e2a4b6bc.
TARGET PORYGON-LIVE-DEMO2

SEVERITY	CONFIDENCE	DISTANCE
0.74	0.92	0.27

POR-DET-006 POR-DET-002 POR-DET-004 POR-DET-005

Trace pipeline: raw event → rule → incident →

HUMAN-IN-THE-LOOP LEASE

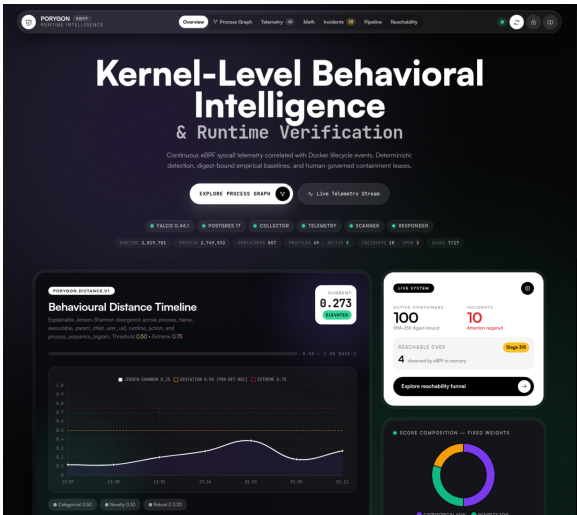
Observe Pause Stop

Deterministic Rules Matrix (v1)

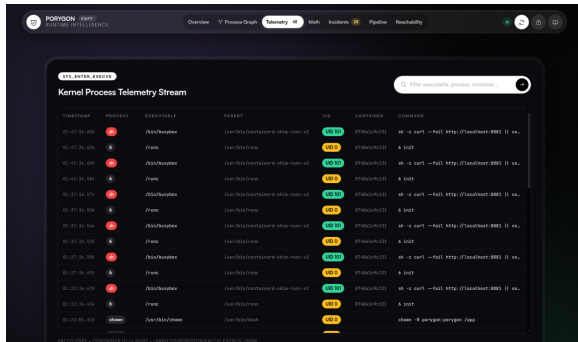
POR-DET-001	High behavioural distance	SEV 0.65 CONF 0.78
POR-DET-002	Previously unseen shell execution	SEV 0.72 CONF 0.98
POR-DET-003	Novel root process	SEV 0.78 CONF 0.98
POR-DET-004	Previously unseen dual-use tool	SEV 0.64 CONF 0.88
POR-DET-005	Shell-to-tool sequence	SEV 0.98 CONF 0.95
POR-DET-006	Docker exec activity	SEV 0.35 CONF 0.35

Live run: severity 0.74, confidence 0.92, distance 0.27; rules POR-DET-002/004/005/006, incident open.

30. Results: System Overview and Raw Telemetry



Live overview: score 0.273, elevated, 10 incidents tracked.



Raw kernel telemetry stream: the source events behind every score.

31. Key Findings

- 1 Exact-image identity is what removed false alarms, not configuration context.
- 2 Adding runtime configuration on top of exact-image identity changed nothing measurable here.
- 3 Why: the tested change (a dropped Linux capability) does not alter *which processes run*, and process-name features cannot see it by construction.
- 4 A follow-on feature scoring the configuration document directly separated every tested case (0/26 false alarms, 27/27 detections): **exploratory, hand-fixed weights, not confirmatory.**

Takeaway

The measured improvement comes from per-image scoping, not from configuration context.

32. Limitations and Future Work

Limitations

- Tested workloads and scenarios only.
- `proc.cmdline` now resolves multicall identity for detection (Slide 26), but command-line arguments are still never a *scoring* feature.
- The broadened POR-DET-004 (Slide 27) is verified against four live-rerun deviations, not yet re-run against the full 216-trial pilot protocol.
- A deviation with genuinely no distinguishable process-level signature (no new executable, no new parent-child edge) would still leave the rule engine with nothing to key on.

Future Work

- Larger, more diverse workload corpus.
- Richer runtime features (syscall arguments).
- Additional distance measures.
- A configuration variant that changes process behaviour.
- Longer observation periods.

33. Technical Enrichment Activity Choices

Professional Online Certification Details (Individual Mandatory: 10–20 Hours)

S.No	Student Name	Chosen Course	Platform	Hours
1	Ashin Varghese	Introduction to Kubernetes (LFS158)	edX	20
2	Anurup Krishna	Introduction to Kubernetes (LFS158)	edX	20
3	Krishnan S	Introduction to Kubernetes (LFS158)	edX	20
4	Madhav Sreejith	Introduction to Kubernetes (LFS158)	edX	20

Requirement: each student completes one approved 10–20 hour certification relevant to the project.

References I

-  J. Lin, "Divergence measures based on the Shannon entropy," *IEEE Transactions on Information Theory*, vol. 37, no. 1, pp. 145–151, Jan. 1991.
-  S. Forrest, S. A. Hofmeyr, A. Somayaji, and T. A. Longstaff, "A sense of self for Unix processes," in *Proc. IEEE Symposium on Security and Privacy*, 1996, pp. 120–128.
-  D. M. Endres and J. E. Schindelin, "A new metric for probability distributions," *IEEE Transactions on Information Theory*, vol. 49, no. 7, pp. 1858–1860, Jul. 2003.
-  J. K. Hoyos-Osorio and L. G. Sanchez-Giraldo, "The representation Jensen–Shannon divergence," in *Proc. 37th Conf. Neural Information Processing Systems (NeurIPS)*, 2023.
-  G. R. Castanhel, T. Heinrich, F. Ceschin, and C. A. Maziero, "Taking a peek: an evaluation of anomaly detection using system calls for containers," in *Proc. IEEE Symp. Computers and Communications (ISCC)*, 2021.
-  A. El Khairi, M. Caselli, C. Knierim, A. Peter, and A. Continella, "Contextualizing system calls in containers for anomaly-based intrusion detection," in *Proc. ACM Cloud Computing Security Workshop (CCSW)*, 2022, pp. 9–21.
-  M. A. Bouke et al., "Multi-level distributional entropy for explainable network intrusion detection," arXiv:2606.29797, 2026.
-  G. Fournier, S. Afchain, and S. Baubeau, "Runtime security monitoring with eBPF," in *Proc. Symp. sur la Sécurité des Technologies de l'Information et des Communications (SSTIC)*, 2021.

References II



X. Lin, Z. Chen, W. Yang, X. Yang, J. Luo, and X. Yang, "eBPF-Guard: a detection method for container escape via multi-level monitoring and enhanced analysis model," *Empirical Software Engineering*, vol. 31, article 51, 2026.



Li Wei, Yuan Zekun, Wu Kehe, and Cheng Rui, "Container anomaly detection based on attention mechanism and multiscale convolutional neural network," *Journal of Information Security Research*, vol. 11, no. 1, pp. 35–, 2025.



K. Xiong, Z. Wu, and X. Jia, "DeepContainer: a deep learning-based framework for real-time anomaly detection in cloud-native container environments," *Journal of Advanced Computing Systems*, 2025.



The Falco Authors, "Falco: cloud-native runtime security," Cloud Native Computing Foundation. [Online]. Available: <https://falco.org>



Open Container Initiative, "OCI image format specification." [Online]. Available: <https://github.com/opencontainers/image-spec>

Thank You

Questions?